the file contains 50 algorithm tasks create pseudocode for each and display results as a numbered list

Here are pseudocode solutions for all 50 algorithms:

1. Anagram Check

```
FUNCTION isAnagram(str1, str2):
    IF length(str1) ≠ length(str2) THEN RETURN False
    sort characters of str1 → sorted1
    sort characters of str2 → sorted2
    RETURN sorted1 == sorted2
```

2. Decimal to Binary

```
FUNCTION decimalToBinary(n):
    IF n == 0 THEN RETURN "0"
    result ← ""
    WHILE n > 0:
        result ← (n MOD 2) + result
        n ← n DIV 2
    RETURN result
```

3. Leap Year Check

```
FUNCTION isLeapYear(year):
    IF (year MOD 400 == 0) THEN RETURN True
    IF (year MOD 100 == 0) THEN RETURN False
    IF (year MOD 4 == 0) THEN RETURN True
    RETURN False
```

4. Intersection of Two Sorted Arrays

```
FUNCTION intersection(A, B):
    result ← []
    i ← 0, j ← 0
    WHILE i < length(A) AND j < length(B):
        IF A[i] == B[j] THEN
            APPEND A[i] TO result
            i++, j++
        ELSE IF A[i] < B[j] THEN i++
        ELSE j++
    RETURN result
```

5. Remove Punctuation

```
FUNCTION removePunctuation(sentence):
    result ← ""
    FOR each char IN sentence:
        IF char is NOT punctuation THEN
            result ← result + char
    RETURN result
```

6. String Length

```
FUNCTION stringLength(str):
    count ← 0
    FOR each char IN str:
        count ← count + 1
    RETURN count
```

7. Split Sentence into Words

```
FUNCTION splitIntoWords(sentence):
    words ← []
```

```
    currentWord ← ""
    FOR each char IN sentence:
        IF char == " " THEN
            IF currentWord ≠ "" THEN APPEND currentWord TO words
            currentWord ← ""
        ELSE
            currentWord ← currentWord + char
    IF currentWord ≠ "" THEN APPEND currentWord TO words
    RETURN words
```

8. Sort Words Alphabetically

```
FUNCTION sortWords(wordList):
    FOR i FROM 0 TO length(wordList) - 2:
        FOR j FROM 0 TO length(wordList) - i - 2:
            IF wordList[j] > wordList[j+1] THEN
                SWAP wordList[j] AND wordList[j+1]
    RETURN wordList
```

9. Linear Search

```
FUNCTION linearSearch(list, T):
    FOR i FROM 0 TO length(list) - 1:
        IF list[i] == T THEN RETURN i
    RETURN -1
```

10. Count Vowel Frequencies

```
FUNCTION countVowels(paragraph):
    counts ← {a:0, e:0, i:0, o:0, u:0}
    FOR each char IN lowercase(paragraph):
        IF char IN counts THEN
```

```
        counts[char] ← counts[char] + 1
    RETURN counts
```

11. Remove Duplicate IP Addresses

```
FUNCTION uniqueIPs(ipList):
    seen ← empty set
    result ← []
    FOR each ip IN ipList:
        IF ip NOT IN seen THEN
            ADD ip TO seen
            APPEND ip TO result
    RETURN result
```

12. Sum of Main Diagonal

```
FUNCTION diagonalSum(grid, N):
    total ← 0
    FOR i FROM 0 TO N - 1:
        total ← total + grid[i][i]
    RETURN total
```

13. Reverse a String

```
FUNCTION reverseString(str):
    result ← ""
    FOR i FROM length(str) - 1 DOWN TO 0:
        result ← result + str[i]
    RETURN result
```

14. Sum of Even Numbers

```
FUNCTION sumEvens(list):
    total ← 0
    FOR each num IN list:
        IF num MOD 2 == 0 THEN
            total ← total + num
    RETURN total
```

15. Second Largest Element

```
FUNCTION secondLargest(arr):
    largest ← second ← -INFINITY
    FOR each num IN arr:
        IF num > largest THEN
            second ← largest
            largest ← num
        ELSE IF num > second AND num ≠ largest THEN
            second ← num
    RETURN second
```

16. GCD (Euclidean Algorithm)

```
FUNCTION gcd(a, b):
    WHILE b ≠ 0:
        temp ← b
        b ← a MOD b
        a ← temp
    RETURN a
```

17. Sum and Average of a List

```
FUNCTION sumAndAverage(list):
    total ← 0
```

```
  FOR each num IN list:
    total ← total + num
  average ← total / length(list)
  RETURN total, average
```

18. Character Frequency Count

```
FUNCTION charFrequency(str):
  freq ← empty dictionary
  FOR each char IN str:
    IF char IN freq THEN freq[char]++
    ELSE freq[char] ← 1
  RETURN freq
```

19. Swap Two Variables Without Temp

```
FUNCTION swapWithoutTemp(a, b):
  a ← a + b
  b ← a - b
  a ← a - b
  RETURN a, b
```

20. Find All Divisors

```
FUNCTION findDivisors(n):
  divisors ← []
  FOR i FROM 1 TO n:
    IF n MOD i == 0 THEN
      APPEND i TO divisors
  RETURN divisors
```

21. Even or Odd

```
FUNCTION evenOrOdd(number):
    IF number MOD 2 == 0 THEN
        RETURN "Even"
    ELSE
        RETURN "Odd"
```

22. Factorial

```
FUNCTION factorial(n):
    IF n == 0 OR n == 1 THEN RETURN 1
    RETURN n * factorial(n - 1)
```

23. Score to Grade

```
FUNCTION scoreToGrade(score):
    IF score >= 70 THEN RETURN "Distinction"
    ELSE IF score >= 55 THEN RETURN "Merit"
    ELSE IF score >= 40 THEN RETURN "Pass"
    ELSE RETURN "Fail"
```

24. Password Validator

```
FUNCTION validatePassword(password):
    IF length(password) < 8 THEN RETURN False
    hasDigit ← False
    hasUpper ← False
    FOR each char IN password:
        IF char is digit THEN hasDigit ← True
        IF char is uppercase THEN hasUpper ← True
    RETURN hasDigit AND hasUpper
```

25. Longest Substring Without Repeating Characters

```
FUNCTION longestUniqueSubstring(str):
    seen ← empty dictionary
    maxLen ← 0
    start ← 0
    FOR i FROM 0 TO length(str) - 1:
        IF str[i] IN seen AND seen[str[i]] >= start THEN
            start ← seen[str[i]] + 1
        seen[str[i]] ← i
        maxLen ← MAX(maxLen, i - start + 1)
    RETURN maxLen
```

26. Level-Order Binary Tree Traversal

```
FUNCTION levelOrder(root):
    IF root is NULL THEN RETURN []
    queue ← [root], result ← []
    WHILE queue is not empty:
        level ← [], nextQueue ← []
        FOR each node IN queue:
            APPEND node.value TO level
            IF node.left ≠ NULL THEN APPEND node.left TO nextQueue
            IF node.right ≠ NULL THEN APPEND node.right TO nextQueue
        APPEND level TO result
        queue ← nextQueue
    RETURN result
```

27. Rotate 2D Matrix 90° Clockwise

```
FUNCTION rotateMatrix(matrix, N):
    // Transpose
```

```
  FOR i FROM 0 TO N - 1:
    FOR j FROM i+1 TO N - 1:
      SWAP matrix[i][j] AND matrix[j][i]
  // Reverse each row
  FOR i FROM 0 TO N - 1:
    REVERSE matrix[i]
  RETURN matrix
```

28. Sliding Window Maximum

```
FUNCTION slidingWindowMax(arr, k):
  result ← []
  deque ← []   // stores indices
  FOR i FROM 0 TO length(arr) - 1:
    WHILE deque not empty AND deque.front < i - k + 1:
      REMOVE front of deque
    WHILE deque not empty AND arr[deque.back] < arr[i]:
      REMOVE back of deque
    APPEND i TO back of deque
    IF i >= k - 1 THEN APPEND arr[deque.front] TO result
  RETURN result
```

29. Text Classification Model

```
FUNCTION classifyText(text, categories, trainingData):
  preprocessed ← tokenize(lowercase(removeStopWords(text)))
  features ← extractFeatures(preprocessed)   // e.g. TF-IDF
  FOR each category IN categories:
    score[category] ← computeSimilarity(features, trainingData[category])
  RETURN category with highest score
```

30. Topic Modelling System

```
FUNCTION extractTopics(corpus, numTopics):
    tokens ← tokenize and clean each document in corpus
    vocabulary ← buildVocabulary(tokens)
    termMatrix ← buildDocumentTermMatrix(tokens, vocabulary)
    topics ← initializeRandomTopicAssignments(numTopics)
    REPEAT until convergence:
        FOR each document:
            FOR each word:
                UPDATE topic assignment using probability distribution
                UPDATE topic-word and document-topic counts
    RETURN top N words per topic as topic descriptions
```

31. Co-reference Resolution

```
FUNCTION resolveCoReferences(text):
    entities ← extractNamedEntities(text)
    mentions ← extractMentions(text)   // nouns and pronouns
    FOR each pronoun IN mentions:
        candidates ← findNearestEntities(pronoun, entities)
        bestMatch ← scoreAndRank(candidates, pronoun)
        MAP pronoun TO bestMatch
    RETURN coreferenceMap
```

32. Question Answering System

```
FUNCTION answerQuestion(question, context):
    qTokens ← tokenize(question)
    cTokens ← tokenize(context)
    qEmbedding ← embed(qTokens)
    FOR each sentence IN context:
        score[sentence] ← similarity(qEmbedding, embed(sentence))
```

```
    bestSentence ← sentence with highest score
    answer ← extractAnswerSpan(question, bestSentence)
    RETURN answer
```

33. Integer Square Root (Binary Search)

```
FUNCTION intSqrt(n):
    IF n == 0 THEN RETURN 0
    low ← 1, high ← n
    WHILE low <= high:
        mid ← (low + high) DIV 2
        IF mid * mid == n THEN RETURN mid
        ELSE IF mid * mid < n THEN
            result ← mid
            low ← mid + 1
        ELSE high ← mid - 1
    RETURN result
```

34. Union of Two Lists (No Duplicates)

```
FUNCTION findCommonElements(list1, list2):
    set1 ← convert list1 to set
    result ← []
    FOR each item IN list2:
        IF item IN set1 AND item NOT IN result THEN
            APPEND item TO result
    RETURN result
```

35. LRU Cache

```
FUNCTION LRUCache(capacity):
    cache ← ordered dictionary
```

```
  FUNCTION get(key):
    IF key NOT IN cache THEN RETURN -1
    MOVE key to most recently used position
    RETURN cache[key]

  FUNCTION put(key, value):
    IF key IN cache THEN REMOVE key
    ELSE IF length(cache) == capacity THEN
      REMOVE least recently used (front) item
    INSERT key:value at most recently used position
```

36. Binary Search in Rotated Sorted Array

```
FUNCTION searchRotated(arr, target):
  low ← 0, high ← length(arr) - 1
  WHILE low <= high:
    mid ← (low + high) DIV 2
    IF arr[mid] == target THEN RETURN mid
    IF arr[low] <= arr[mid] THEN   // left half is sorted
      IF arr[low] <= target < arr[mid] THEN high ← mid - 1
      ELSE low ← mid + 1
    ELSE                           // right half is sorted
      IF arr[mid] < target <= arr[high] THEN low ← mid + 1
      ELSE high ← mid - 1
  RETURN -1
```

37. Stack Using Array

```
FUNCTION Stack():
  data ← []
```

```
  FUNCTION push(item):
    APPEND item TO data

  FUNCTION pop():
    IF data is empty THEN RETURN "Underflow"
    REMOVE and RETURN last item of data

  FUNCTION peek():
    IF data is empty THEN RETURN "Empty"
    RETURN last item of data
```

38. First Non-Repeating Character

```
FUNCTION firstNonRepeating(str):
  freq ← empty dictionary
  FOR each char IN str:
    freq[char] ← freq.get(char, 0) + 1
  FOR each char IN str:
    IF freq[char] == 1 THEN RETURN char
  RETURN None
```

39. Detect Cycle in Linked List (Floyd's)

```
FUNCTION hasCycle(head):
  slow ← head
  fast ← head
  WHILE fast ≠ NULL AND fast.next ≠ NULL:
    slow ← slow.next
    fast ← fast.next.next
    IF slow == fast THEN RETURN True
  RETURN False
```

40. Queue Using Two Stacks

```
FUNCTION Queue():
  stack1 ← [], stack2 ← []

  FUNCTION enqueue(item):
    PUSH item ONTO stack1

  FUNCTION dequeue():
    IF stack2 is empty THEN
      WHILE stack1 is not empty:
        PUSH POP(stack1) ONTO stack2
    IF stack2 is empty THEN RETURN "Underflow"
    RETURN POP(stack2)
```

41. N-Queens Problem

```
FUNCTION solveNQueens(N):
  board ← N×N grid of 0s
  CALL placeQueens(board, 0, N)

FUNCTION placeQueens(board, row, N):
  IF row == N THEN RECORD solution; RETURN
  FOR col FROM 0 TO N - 1:
    IF isSafe(board, row, col, N) THEN
      board[row][col] ← 1
      placeQueens(board, row + 1, N)
      board[row][col] ← 0   // backtrack

FUNCTION isSafe(board, row, col, N):
  CHECK column, upper-left diagonal, upper-right diagonal for queens
  RETURN True if no conflicts
```

42. Pairs Summing to Target

```
FUNCTION findPairs(arr, target):
    seen ← empty set
    pairs ← []
    FOR each num IN arr:
        complement ← target - num
        IF complement IN seen THEN
            APPEND (complement, num) TO pairs
        ADD num TO seen
    RETURN pairs
```

43. Tower of Hanoi

```
FUNCTION hanoi(n, source, destination, auxiliary):
    IF n == 1 THEN
        PRINT "Move disc 1 from " + source + " to " + destination
        RETURN
    hanoi(n - 1, source, auxiliary, destination)
    PRINT "Move disc " + n + " from " + source + " to " + destination
    hanoi(n - 1, auxiliary, destination, source)
```

44. Maximum Subarray Sum (Kadane's Algorithm)

```
FUNCTION maxSubarraySum(arr):
    maxSum ← arr[0]
    currentSum ← arr[0]
    FOR i FROM 1 TO length(arr) - 1:
        currentSum ← MAX(arr[i], currentSum + arr[i])
        maxSum ← MAX(maxSum, currentSum)
    RETURN maxSum
```

45. Matrix Addition

```
FUNCTION addMatrices(A, B, rows, cols):
    result ← rows × cols matrix of 0s
    FOR i FROM 0 TO rows - 1:
        FOR j FROM 0 TO cols - 1:
            result[i][j] ← A[i][j] + B[i][j]
    RETURN result
```

46. Matrix Multiplication

```
FUNCTION multiplyMatrices(A, B, n, m, p):
    // A is n×m, B is m×p
    result ← n × p matrix of 0s
    FOR i FROM 0 TO n - 1:
        FOR j FROM 0 TO p - 1:
            FOR k FROM 0 TO m - 1:
                result[i][j] ← result[i][j] + A[i][k] * B[k][j]
    RETURN result
```

47. Inventory Update

```
FUNCTION updateInventory(inventory, transactions):
    FOR each transaction IN transactions:
        item ← transaction.item
        type ← transaction.type
        quantity ← transaction.quantity
        IF item NOT IN inventory THEN inventory[item] ← 0
        IF type == "purchase" THEN
            inventory[item] ← inventory[item] - quantity
        ELSE IF type == "delivery" THEN
```

```
        inventory[item] ← inventory[item] + quantity
    IF inventory[item] < 0 THEN ALERT "Insufficient stock for " + item
  RETURN inventory
```

48. Recursive File Search

```
FUNCTION findFilesByType(folder, targetExtension):
  results ← []
  FOR each item IN folder.contents:
    IF item is a file AND item.extension == targetExtension THEN
      APPEND item.path TO results
    ELSE IF item is a folder THEN
      results ← results + findFilesByType(item, targetExtension)
  RETURN results
```

49. Maze Pathfinding (BFS)

```
FUNCTION solveMaze(maze, start, end):
  queue ← [(start, [start])]
  visited ← {start}
  WHILE queue is not empty:
    (current, path) ← DEQUEUE from queue
    IF current == end THEN RETURN path
    FOR each neighbour OF current (up/down/left/right):
      IF neighbour is valid AND not a wall AND not visited THEN
        ADD neighbour TO visited
        ENQUEUE (neighbour, path + [neighbour]) TO queue
  RETURN "No path found"
```

50. Drone Collision Avoidance Coordination

```
FUNCTION coordinateDrones(drones, airspace):
    WHILE mission not complete:
        FOR each drone IN drones:
            nextMove ← drone.planNextMove(drone.destination)
            FOR each otherDrone IN drones:
                IF predictedCollision(drone, otherDrone, nextMove) THEN
                    nextMove ← drone.rerouteAround(otherDrone)
            IF nextMove is within airspace bounds THEN
                drone.move(nextMove)
            ELSE
                drone.hover()
            drone.broadcastPosition(drone.currentPosition)
        SYNCHRONISE all drone positions
```